

Software Engineering

How software is conceived, developed, integrated, deployed, operated and evolved.

How software is built

A software product usually evolves from a need into requirements, solution design, development, testing, delivery, operation and maintenance. QA can contribute from requirement analysis through production observation.

SDLC and STLC

SDLC means Software Development Life Cycle. STLC means Software Testing Life Cycle. The first describes the development lifecycle; the second organizes testing activities such as analysis, planning, design, preparation, execution and closure.

Functional and non-functional requirements

Functional requirements describe what the system must do. Non-functional requirements cover attributes such as performance, security, accessibility, reliability and usability. QA should challenge ambiguity and turn requirements into verifiable conditions.

Architectures QA encounters

Monoliths, layered architectures, client-server, microservices, event-driven systems and serverless change where failures can occur. Understanding components and dependencies improves investigation.

Frontend, backend and integrations

Frontend is the user interaction layer; backend executes rules and processing; APIs, queues and events connect services. A defect visible on screen may originate in any of these layers.

Architectural quality

Performance, security, reliability, maintainability, compatibility and scalability should be considered from design time. Testability and observability also influence the ability to detect and diagnose problems.

Technical debt, refactoring and maintenance

Technical debt represents decisions that speed up delivery but may increase future cost. Refactoring improves internal structure without intending to change behavior; it can still require risk-based regression.

Advanced QA perspective

An experienced QA does not need to be an architect, but should be able to read flows, identify failure points and discuss risks, dependencies, contracts, data and recovery mechanisms.

QA tip

When investigating a problem, draw the data flow before assuming the cause is in the UI.